

Project Documentation: SmartCourse

Team Lead: Shachar Tsrafati (318790169)

Part A: User Process Description

This section outlines the system processes from a user perspective, reflecting the precise logic implemented in the source code.

Scenario 1: Multiple Teacher Search

- **Scenario Name:** Searching for a teacher that returns multiple results.
- **Prerequisites:** * The user is logged in as a **Student**.
 - There are at least two teachers named "**John**" in the database.
- **Step-by-Step:**
 1. Navigate to the **Search Screen**.
 2. Enter the name "**John**" into the search field.
 3. Click the **Search** button.
- **Expected Result:** The system displays a list containing all teachers whose names match "John."

Scenario 2: Search for Non-Existent Name

- **Scenario Name:** Searching for a name that does not appear in the database.
- **Prerequisites:** The user is currently on the **Search Screen**.
- **Step-by-Step:**
 1. Enter a name that does not exist in the system, e.g., "**UnknownName**".
 2. Click the **Search** button.
- **Expected Result:** The results list remains empty, and no users are displayed.

Scenario 3: Partial Matching and Name Exclusion

- **Scenario Name:** Testing search precision and the exclusion of similar names.
- **Prerequisites:** There is one teacher named "**Orian**" and another teacher named "**Ori**" in the system.
- **Step-by-Step:**
 1. Enter the string "**Ori**" into the search field.
 2. Click the **Search** button.
- **Expected Result:** The system displays only "**Orian**" (consistent with the server's response logic) and ensures that "**Ori**" does not appear by mistake if it was not explicitly returned in the results.

```

@OptIn(ExperimentalCoroutinesApi::class)
class SearchUserViewModelTest {

    private val testDispatcher = StandardTestDispatcher()

    @Mock
    private lateinit var userRepository: UserRepository
    @Mock
    private lateinit var authRepository: AuthRepository
    private lateinit var viewModel: SearchUserViewModel

    @Before
    fun setup() {
        MockitoAnnotations.openMocks(this)
        Dispatchers.setMain(testDispatcher)

        // Initial setup: current user is a student
        val me = User(userId = "my_user_id", name = "Me", role = UserRole.STUDENT)

        runTest(testDispatcher) {
            whenever(authRepository.restoreValidSession()).thenReturn(me)
            whenever(userRepository.getUserCourses(anyString())).thenReturn(emptyList())
        }

        viewModel = SearchUserViewModel(userRepository, authRepository)
        testDispatcher.scheduler.advanceUntilIdle()
    }

    @After
    fun tearDown() {
        Dispatchers.resetMain()
    }

    /**
     * Test 1: Student searching for multiple tutors.
     * Checks if the filter allows multiple valid tutors to be displayed.
     */
    @Test
    fun `onQueryChanged returns multiple tutors for student search`() = runTest {
        val query = "John"
        val tutor1 = User("t1", "John Smith", role = UserRole.TUTOR)
        val tutor2 = User("t2", "John Doe", role = UserRole.TUTOR)

        whenever(userRepository.searchUsers(query)).thenReturn(listOf(tutor1, tutor2))

        viewModel.onQueryChanged(query)
        advanceUntilIdle()

        val state = viewModel.uiState.value
        assertEquals(2, state.results.size)
        println("Test 1: Found ${state.results.size} tutors.")
    }

    /**
     * Test 2: Search for a name that does not exist in the database.
     * Checks if the UI state remains empty when the repository returns nothing.
     */
    @Test
    fun `onQueryChanged returns empty list when no users exist`() = runTest {
        val query = "UnknownName"
        whenever(userRepository.searchUsers(query)).thenReturn(emptyList())

        viewModel.onQueryChanged(query)
        advanceUntilIdle()

        val state = viewModel.uiState.value
        assertEquals(0, state.results.size)
        println("Test 2: No users found for query: $query")
    }

    /**
     * Test 3: Verifies specific string matching and exclusion logic.
     */
    @Test
    fun `onQueryChanged handles specific string match correctly`() = runTest {
        // 1. Setting up conditions [cite: 35]
        val query = "Ori"
        val orian = User("u1", "Orian", role = UserRole.TUTOR)
        // Repository returns "Orian" for query "Ori"
        whenever(userRepository.searchUsers(query)).thenReturn(listOf(orian))

        // 2. Calling the function
        viewModel.onQueryChanged(query)
        advanceUntilIdle()

        // 3. Assertions [cite: 37]
        val state = viewModel.uiState.value
        assertEquals(1, state.results.size)
        assertEquals("Orian", state.results[0].user.name)

        // Ensure "Ori" is NOT in the results (if not returned by repo)
        val containsOri = state.results.any { it.user.name == "Ori" }
        assertEquals(false, containsOri)
    }
}

```

Part B: Technical Test Cases (Unit Testing)

These tests follow the **AAA (Arrange, Act, Assert)** pattern to ensure the SearchViewModel correctly manages the UI state based on repository responses.

1. Verify Multiple Results Handling

Objective: Ensure that when a search yields multiple matches, the ViewModel updates the state with the correct number of items.

- **Arrange:** Mock the repository to return a list containing two lecturers—"John Smith" and "John Doe"—when the query "John" is provided.
- **Act:** Invoke the onQueryChanged function (or search trigger) with the value "John".
- **Assert:** Verify that the state.results list within the ViewModel contains exactly 2 items.

2. Verify Search with No Results

Objective: Ensure the system handles non-existent names gracefully and clears previous state.

- **Arrange:** Mock the repository to return an **empty list** when searching for a non-existent name like "UnknownName".
- **Act:** Execute the search logic in the ViewModel using the string "UnknownName".
- **Assert:** Verify that the state.results size is 0. This ensures the UI will correctly display an "Empty State" or "No Results Found" message.

3. Verify Search Precision and Exclusion

Objective: Confirm that the ViewModel strictly displays only the data returned by the data source and does not introduce "ghost" matches or partial artifacts.

- **Arrange:** Configure the mock repository to return exactly one result, "Orion", for the search string "Ori".
- **Act:** Trigger the search process in the ViewModel for "Ori".
- **Assert:**
 - **Content Check:** Confirm that exactly one result is returned and its name is strictly "Orion".
 - **Exclusion Check:** Verify that the search string itself ("Ori") does not appear as a standalone result unless explicitly provided by the repository. This prevents the UI from displaying "junk" or placeholder data.

Part C:

```
@RunWith(AndroidJUnit4::class)
class SearchUITest {

    @get:Rule
    val composeTestRule = createComposeRule()

    private val userRepository = mockk<UserRepository>(relaxed = true)
    private val authRepository = mockk<AuthRepository>(relaxed = true)
    private val me = User(userId = "me", name = "Me", role = UserRole.STUDENT)

    @Before
    fun setup() {
        coEvery { authRepository.restoreValidSession() } returns me
        coEvery { userRepository.getUserCourses(any()) } returns emptyList()
    }

    private fun launchSearchScreen(query: String, results: List<User>, expectedResultName: String? =
null) {
        coEvery { userRepository.searchUsers(query) } returns results

        val viewModel = SearchUserViewModel(userRepository, authRepository)
        composeTestRule.setContent {
            SearchUserScreen(navController = rememberNavController(), viewModel = viewModel)
        }

        // Action: Type the search query
        composeTestRule.onNode(hasSetTextAction()).performTextInput(query)

        // Fix: If we expect a result, wait until it is actually displayed (handles debounce/async)
        if (expectedResultName != null) {
            composeTestRule.waitUntil(timeoutMillis = 5000) {
                composeTestRule
                    .onAllNodesWithText(expectedResultName)
                    .fetchSemanticsNodes().isNotEmpty()
            }
        }
    }

    @Test
    fun testMultipleResultsSearchFlow() {
        val tutors = listOf(
            User("t1", "John Smith", role = UserRole.TUTOR),
            User("t2", "John Doe", role = UserRole.TUTOR)
        )
        // We pass "John Smith" so the test waits for it to appear
        launchSearchScreen("John", tutors, expectedResultName = "John Smith")

        composeTestRule.onNodeWithText("John Smith").assertIsDisplayed()
        composeTestRule.onNodeWithText("John Doe").assertIsDisplayed()
    }

    @Test
    fun testEmptyResultsSearchFlow() {
        launchSearchScreen("UnknownName", emptyList())
        composeTestRule.onNodeWithText("John Smith").assertDoesNotExist()
    }

    @Test
    fun testSpecificStringMatchFlow() {
        // We pass "Orlan" so the test waits for it to appear
        launchSearchScreen("Ori", listOf(User("u1", "Orlan", role = UserRole.TUTOR)),
            expectedResultName = "Orlan")

        composeTestRule.onNodeWithText("Orlan").assertIsDisplayed()
        composeTestRule.onNodeWithText("Or").assertDoesNotExist()
    }
}
```

UI and Integration Test Flows

1. Multiple Results Search Flow

Method Name: testMultipleResultsSearchFlow

Objective: Verify that the system correctly renders a list when the search query returns more than one user.

- **Arrange:** Define a mock environment where two distinct lecturers exist with the name "John" (e.g., "John Smith" and "John Doe"). Configure the Repository to return both entities when a search call for "John" is triggered.
- **Act:** Simulate the user typing the string "John" into the search text field on the UI.
- **Assert:** Confirm that both names appear on the screen simultaneously. This ensures the LazyColumn (or equivalent list component) is rendering items correctly and that data is not being overwritten during the update process.

2. Empty Results Search Flow

Method Name: testEmptyResultsSearchFlow

Objective: Ensure a consistent user experience and UI integrity when no matches are found in the system.

- **Arrange:** Configure a scenario where a search for a non-existent name (e.g., "UnknownName") is performed. Instruct the Repository to return an empty list.
- **Act:** Simulate the user typing the non-existent name into the search field.
- **Assert:** Verify that previously displayed results (such as "John Smith") are removed from the screen. This confirms that the screen correctly clears the state and displays the appropriate "Empty" or "No Results" view.

3. Precision and Exclusion Flow

Method Name: testSpecificStringMatchFlow

Objective: Validate that the search is precise and strictly displays only the results provided by the official data source without artifacts.

- **Arrange:** Create a specific case where a search for "Ori" is performed, and the repository is set to return only "Orian".
- **Act:** Simulate the user typing "Ori" into the search field.
- **Assert:** Perform a dual-verification:
 1. **Inclusion:** Verify that "Orian" is visible (as it contains the search string).
 2. **Exclusion:** Verify that shorter strings like "Or" are not displayed, even if they are substrings of the result.

Code Review Report: Part D

I am one man team so I will review 3 files

Project: Mealy

File Reviewed: RecipeAdapter

Team Lead ID: 211724034

General Overview

The RecipeAdapter implements a ListAdapter to display recipe items. Each item includes textual information, a CheckBox for favorites, and a **Nested RecyclerView** to preview up to 8 ingredients. While using ListAdapter is a strong architectural choice for efficiency, there are critical implementation details regarding view recycling and state management that require attention.

Pros and Cons Analysis

Feature	Status	Detail
ListAdapter Usage	Pro	A correct choice that allows background Diffing calculations, preventing UI flickering during updates.
Interface Separation	Pro	Listeners are defined externally, keeping the Adapter "dumb" (free of business logic) and improving testability.
Preview Management	Pro	Using a limit function to display only 8 ingredients prevents UI clutter and keeps the layout clean.
notifyDataSetChanged	Con	Calling this inside a ListAdapter negates the benefits of DiffUtil, forcing a full re-render of the entire list.
Recycling Bug	Con	Setting listeners in onBind without resetting them leads to logic being triggered on the wrong items during scroll.
Nested RV Performance	Con	Initializing the internal Adapter/LayoutManager in the ViewHolder constructor can cause memory overhead in very long lists.

3 Key Comments

1. **Preventing the "Double-Check" Bug (Listener Recycling):** In a RecyclerView, Views are reused. When you set an OnCheckedChangeListener in onBindViewHolder, you **must** nullify the listener before updating the CheckBox state. If you don't, the programmatic update of the state during scrolling will

accidentally trigger the onFavorited logic of the previous item that occupied that View.

2. **Optimizing Nested RecyclerViews:** A list within a list is computationally expensive. It is highly recommended to use a shared RecyclerView.RecycledViewPool. This allows all internal adapters to share a common pool of ingredient ViewHolders, significantly improving scroll performance and reducing memory consumption.
3. **Moving to a Reactive State:** Instead of maintaining a manual list of favorites and calling notifyDataSetChanged, the isFavorite property should be part of the Recipe Data Class itself. This way, any change is handled via the standard submitList() method, allowing DiffUtil to update only the specific item that changed.

Example of Improvement

The Problem: The favorite listener may be triggered accidentally during scrolling (Recycling).

The Solution: Reset the listener to null before updating the UI state, then re-attach it.

Placement: Inside onBindViewHolder within the RecipeAdapter.

Project: Mealy **File Reviewed:** KitchenItem **Team Lead ID:** 211724034

General Overview

The KitchenItem class serves as the primary data model for pantry inventory. It integrates with Firebase Firestore via the @DocumentId annotation and implements Java's Cloneable and Serializable interfaces. While the class is concise and readable, it contains structural risks regarding the equality contract and object deep-copying that could lead to data corruption in lists or maps.

Pros and Cons Analysis

Feature	Status	Detail
Readability	Pro	Short, descriptive variable names and clear static factory methods like toKey make the code easy to follow.
Firebase Integration	Pro	The use of @DocumentId simplifies the mapping between local objects and remote Firestore documents.
Null Safety	Pro	Use of @NonNull and @Nullable annotations provides compile-time safety and clear documentation for developers.
HashCode Contract	Con	The hashCode method only considers the ingredientKey field, while equals considers both the key and quantity. This violates the Java contract.

Feature	Status	Detail
Shallow Cloning	Con	The clone() method performs a shallow copy of the Quantity object. Modifying the quantity of a clone will inadvertently modify the original.
Getter Inconsistency	Con	The fallback logic in getIngredientKey is not utilized within hashCode, leading to unpredictable behavior if the object is identified by its documentId.

3 Key Comments

1. **Synchronizing Equals and hashCode:** In Java, if two objects are equal according to the equals(Object) method, they must have the same hashCode(). Currently, changing a Quantity changes the equality result but not the hash code. This will cause KitchenItem objects to "disappear" or behave incorrectly when stored in a HashMap or HashSet. Both methods must reference the same set of fields.
2. **Implementing Deep Copy for Mutability:** Since Quantity is a mutable object, the clone() method must perform a deep copy. Without this, the original and the cloned KitchenItem share the same memory reference for their quantity. Any logic that simulates "what-if" scenarios (e.g., checking if enough stock remains after a recipe) will accidentally subtract from the actual inventory state.
3. **Standardizing Identity Logic:** The class provides a fallback where getIngredientKey() returns the documentId if the key is null. However, the internal logic of the class often bypasses this getter to access the fields directly. To ensure consistent behavior regardless of whether the item was created locally or pulled from Firestore, all internal comparisons should use the public getter methods.

Example of Improvement

The Problem: The hashCode is inconsistent with equals, and the clone method does not protect the Quantity data from shared-reference mutations.

The Solution: Use Objects.hash() to include all relevant fields and explicitly call clone() on the quantity object during the cloning process.

Placement: Inside the KitchenItem class, overriding the default hashCode and clone methods.

Project: Mealy

File Reviewed: NotificationRepo

Team Lead ID: 211724034

General Overview

The NotificationRepo manages the lifecycle of user notifications using Firebase Firestore. It utilizes WriteBatch for group updates and returns reactive data via LiveData. The primary concerns in this module involve the lack of error handling for asynchronous

operations and the absence of a "safe mechanism" or local caching to prevent unnecessary network overhead for repetitive Insert/Delete actions.

Pros and Cons Analysis

Feature	Status	Detail
LiveData Integration	Pro	Using FirestoreQueryLiveData ensures the UI stays synchronized with the database in real-time without manual polling.
Batch Processing	Pro	The setStatusFor method correctly uses WriteBatch, which is more efficient and atomic than individual updates for list operations.
Clean API	Pro	The repository abstracts Firestore's collection path complexity, providing a simple interface for the rest of the application.
Missing Error Listeners	Con	Methods like insert, delete, and batch.commit() do not implement success or failure listeners, making the app fail silently on network errors.
Hardcoded Paths	Con	Repeatedly typing "users" and "notifications" strings increases the risk of typos. These should be defined as constants.
Redundant Network Calls	Con	There is no logic to prevent duplicate notifications or to verify if an item exists before attempting a deletion.

3 Key Comments

1. **Implementing "Idempotent" Inserts:** To avoid the "Delete/Insert" cycle or duplicate notifications, you should use a specific ID (like a hash of the content and timestamp) instead of calling .add(). By using .set(), if the notification already exists, it simply overwrites it rather than creating a duplicate. This acts as a safe mechanism to ensure data integrity even if the insert logic is triggered multiple times.
2. **Asynchronous Feedback Loop:** Currently, the insert and delete methods are "fire and forget." In a production environment, you should return a Task<Void> or use a callback. Without this, the UI cannot show a loading spinner or an error Toast if the database write fails due to permissions or connectivity issues.
3. **Batch Size Limitations:** WriteBatch has a limit of 500 operations in Firestore. While notifications for a single user might not exceed this often, the setStatusFor method does not check the list size. If the list contains 501 items, the entire operation will crash. Implementing a simple counter to split batches would improve the robustness of the system.

Example of Improvement

The Problem: The current insert method uses `.add()`, which creates a new random document ID every time, potentially leading to duplicate notifications.

The Solution: Use a unique identifier for the notification document and utilize `.set()` with `SetOptions.merge()`. This ensures that if the notification is sent twice, it only exists once in the database.

Placement: Inside the insert method of NotificationRepo.